Guia de estudo — Operação Sentinel (situação em 02/11/2026)

Leia nesta ordem. Cada item diz **o que é**, **onde está** e **o que você precisa saber explicar**.

0. Onde estamos no cronograma

Etapa (enunciado)	Situação	Entrega	Arquivo
Hora 1 — Briefing e desenho	✔	A	`docs/A_arquitetura_e_spof.md`
Hora 2 — Topologia	✔	B	`docker-compose.yml`, `docs/B_topologia_compose.md`
Hora 3 — Rust e health checks	✔	B	`rust-app/src/main.rs`
Hora 4 — Nginx e balanceamento	✔	C	`nginx/nginx.conf`, `docs/C_nginx_fluxo_requisicao.md`
Hora 5 — Watchdog e observabilidade	✔	D	`watchdog/watchdog.py`, `prometheus/*.yml`, `docs/D_plano_monitoramento.md`
Apresentação preliminar 05/11	**material pronto**	H	`docs/apresentacao/`
Hora 6 — Experimentos	◆ **parcial** E1, E5, E7 feitos; E2–E4, E6 previstos	E	`docs/E_matriz_experimentos.md`
Hora 7 — Incidente surpresa	previsto em aula	F	`docs/F_relatorio_incidente.md` (preparação)
Hora 8 — Análise e entrega	previsto	F, G	`README.md` (já utilizável)
Demo final 12/11	roteiro pronto	H	`scripts/demo_final.sh`

1. Estrutura do repositório

```
docker-compose.yml          5 serviços, 1 rede, portas mínimas, healthchecks, restart limitado
nginx/nginx.conf            proxy: round-robin, timeouts 1s/3s, retry ≤2, logs com request_id
rust-app/src/main.rs        rotas /, /erro, /health/live, /health/ready, /metrics, /admin/*
rust-app/Dockerfile         build em 2 estágios (rust:1.90 → debian slim + curl)
watchdog/watchdog.py        sondas 5s/2s/3 falhas, teste sintético, :9100/metrics
prometheus/prometheus.yml   coleta a cada 5 s: rust-a, rust-b, watchdog, prometheus
prometheus/alertas.yml      3 alertas obrigatórios + 5 complementares
scripts/teste_sintetico.sh  N requisições pelo Nginx com contagem por nó/status
scripts/admin.sh            ready/atraso/live/check/metrics a partir de DENTRO da rede
scripts/promq.py            consultas ao Prometheus (query, alerts, targets, rules)
scripts/evidencias_preliminar.sh  gera evidencias/preliminar/01..09
scripts/medir_retorno.sh    tempo até o tráfego voltar a um nó após start
scripts/medir_relogio.py    saltos do relógio de parede
scripts/demo_final.sh       demonstração de 12/11 com pausas e previsões
docs/                       entregas A–F, achados, guia de logs, apresentação
evidencias/preliminar/      saídas reais com horário (02/11/2026)
```

2. Os 6 conceitos que vão perguntar (e a nossa prova de cada um)

Conceito	Explicação em uma frase	Prova no nosso sistema
SPOF	componente cuja queda derruba o serviço	Nginx, watchdog (monitoramento), Prometheus (alertas), host — `docs/A §3`
Liveness × readiness × sintético	vivo? / pode receber tráfego? / o caminho do usuário funciona?	`03_health_checks.txt`: live 200 com ready 503; alvo `sintetico-nginx` no watchdog
Timeout + retry limitado	esperar pouco e repetir uma vez, só o que é seguro repetir	`05_…`: `upstream_status=504, 200` com `req_time=1.001`
Falhas consecutivas	não decidir com uma amostra	watchdog: `falhas_consecutivas=1/3, 2/3, 3/3` → `transicao=DOWN`
SLI → SLO → alerta	número de qualidade → meta → aviso acionável	`docs/D §2 e §4`
Correlação por request_id	o mesmo id atravessa Nginx e Rust	fim de `04_balanceamento_20req.txt`

3. Situações observadas (com números) — decore estes

Situação	Observado	Arquivo
Normal, 20 req	A 10 / B 10, todas 200, ~1 ms	`05` (bloco final), `05b` — em `04` deu B 20 (achado 8)
A não pronto	live 200, ready 503; usuário 10/10 com 200 via B; watchdog DOWN após 3 × 503 (15 s); UP na 1ª sonda OK	`03`
A parado	20/20 com 200 via B; 2–3 req a ~1,09 s (`05b`, `05c`, ensaio); DOWN em 13,5–16,8 s; `NoDown` ativo em ~22 s (`05`); `ColetaFalhando` antes	`05`, `05b`, `05c`
A volta (`start`)	watchdog UP em 2,2–3,5 s; tráfego volta em **até ~10 s** (medido: ≤ 4,6 s, 9,4 s, 10,5 s — depende da fase do ciclo de re-resolução); `RestartCount=0`	`05`, `11`
Config quebrada	`nginx -t` aponta arquivo e linha; Nginx em produção intacto	`06`
Portas	só `*:8080` e `127.0.0.1:9090`; `/metrics` e `/admin` → 404; 9100 sem conexão	`09`
Relógio	saltos −2,4 s / +5,1 s a cada ~30 s	`10`

4. Achados (o que deu "errado" e o que aprendemos)

Ver `docs/achados.md`. Resumo para falar em 20 s:

1. Timeout da sonda não cobria DNS → prazo total na thread.
2. `increase()` extrapolou → flapping falso → `changes()`.
3. Teste sintético conta para o `max_fails` do Nginx.
4. Monitor e balanceador discordam por até ~8 s na volta (DNS `valid=10s`).
5. Nó volta a ficar pronto, mas o Nginx o exclui por até 10 s (`fail_timeout` renovado pela sonda sintética).
6. SLI de erro no nó contou 503 que o usuário nunca viu → corrigido; o certo é medir na borda.

7. Healthcheck do Nginx passava pelos nós → `/nginx/live` local.
8. Relógio do WSL2 salta → correlacionar por `request_id`.

5. Como reproduzir tudo do zero (5 min)

```
docker compose down && docker compose up -d --build
sleep 20 && docker compose ps
scripts/evidencias_preliminar.sh        # ~3 min, regrava evidencias/preliminar/01..09
scripts/medir_retorno.sh rust-a 25      # ~45 s
```

6. Material da apresentação

- Slides (5): `docs/apresentacao/slides_preliminar.md` (Marp → PDF/PPTX).
- Falas, tempos, perguntas prováveis e checklist: `docs/apresentacao/roteiro_fala_e_perguntas.md`.
- Demo final (12/11): `docs/apresentacao/roteiro_demo_final.md` + `scripts/demo_final.sh`.